American International University- Bangladesh

Department of Electrical and Electronic Engineering

EEE 4103: Microprocessor and Embedded Systems Laboratory

Title: Study of a Digital Timer using the millis() function of Arduino to avoid problems associated with the delay() function and power measurement in different power reduction modes while the circuit is in sleep modes.

Introduction:

The objectives of this experiment are to

1. Study the application of the millis() function, a built-in Timer of Arduino.
2. Build a digital timer to turn on several LEDs every minute sequentially on an Arduino Microcontroller Board.
3. Implement a sequential LED light pattern control system using an input switch and an Arduino Microcontroller Board.
4. Measure the amount of power and energy saved using power down or power save mode.

Theory and Methodology:

Up to now, when you've wanted something to happen at a specific time interval with the Arduino, you've used delay(). This is handy, but a little confining. When the Arduino calls the delay() function, it freezes its current state for the entire duration of the delay. That means, there can be no other input or output while it is waiting. Delays are also not very helpful for keeping track of time. If you wanted to do something every 10 seconds, having a 10-second delay would be cumbersome. The millis() function helps to solve these problems. It keeps track of the time your Arduino has been running in milliseconds.

So far, you've been declaring variables as int. An int (integer) is a 16-bit number, it holds values between –32,768 and 32,767 (since the MSB is used for a signed bit, so number ranges are between -2^{15} and $2^{15}-1$). Those may be some large numbers, but if the Arduino is counting 1000 times a second with millis(), you would run out of space in less than a minute. The long data type holds a 32-bit number (between –2,147,483,648 and 2,147,483,647). Since you can't run backward time, to get negative numbers, the variable to store millis() time is called an unsigned long. When a datatype is called unsigned, it is only positive. This allows you to count even higher numbers. An unsigned long number can count up to 4,294,967,295. That is enough space for milis() function to store time for almost 50 days. By comparing the current millis() function to a specific value, you can see if a certain amount of time has passed.

When you turn your Digital Timer over, a tilde switch will change its state, and that will set off another cycle of LEDs turning on. The Tilde switch works just like a regular switch in that it is an on/off sensor. You'll use it here as a digital input. What makes tilt switches unique is that they detect orientation. Typically, they have a small cavity inside the housing that has a metal ball. When tilted in the proper way, the ball rolls to one side of the cavity and connects the two leads that are in your breadboard, closing the switch. With six LEDs, your timer will run for six minutes.

Power Management and Sleep Modes:

Power consumption is a critical issue for a device running continuously for a long time without being turned off. So, to overcome this problem almost every controller comes with a Sleep Mode, which helps developers to design electronic gadgets. Sleep mode puts the device in power-saving mode optimal power consumption by turning off the unused modules. An

© Dept. of EEE, Faculty of Engineering, American International University-Bangladesh (AIUB)

Arduino Sleep mode is also referred to as Arduino Power Save Mode or Arduino Standby Mode. Arduino UNO, Arduino Nano, and Pro-mini come with ATmega328P and it has a Brown-Out Detector (BOD), which monitors the supply voltage at the time of Sleep Mode.

Sleep modes enable the application to shut down unused modules in the MCU, thereby saving power. The AVR provides various sleep modes allowing the user to tailor power consumption to the application's requirements. There are six sleep modes in ATmega328P:

Table 7-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

	Active Clock Domains					Oscillators		Wake-up Sources							
Sleep Mode	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other/O	Software BOD Disable
Idle			X	X	X	X	X(2)	X	X	X	X	X	X	X	
ADC Noise Reduction				X	X	X	X(2)	X(3)	X	X(2)	X	X	X		
Power-down								X(3)	X				X		X
Power-save					X		X(2)	X(3)	X	X			X		X
Standby(1)						X		X(3)	X				X		X
Extended Standby					X(2)	X	X(2)	X(3)	X	X			X		X

Notes: 1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT1 and INT0, only level interrupt.

SMCR – Sleep Mode Control Register

The Sleep Mode Control Register (SMCR) contains control bits for power management. The function of these bits (SM2:0) is shown in the next slide. For entering any of the sleep modes, we need to enable the sleep bit in the Sleep Mode Control Register (SMCR.SE). Then, the Sleep Mode Select bits select any 1 of the sleep modes among 6 different sleep modes, viz. Idle, ADC Noise Reduction, Power-Down, Power-Save, Standby, and External Standby. An internal or external Arduino interrupt or a Reset can wake up the Arduino from sleep mode.

Bit	7	6	5	4	3	2	1	0	
0x33 (0x53)	–	–	–	–	SM2	SM1	SM0	SE	SMCR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 7-2. Sleep Mode Select

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby(1)
1	1	1	External Standby(1)

Note: 1. Standby mode is only recommended for use with external crystals or resonators.

Idle Sleep Mode:

For entering into the Idle sleep mode, write the SM[2,0] bits of the controller '000'. This mode stops the CPU but allows the SPI, 2-wire serial interface, USART, Watchdog, counters, and analog comparator to operate. Idle mode basically stops the CLKCPU and CLKFLASH. Arduino can be woken up at any time by using the external or internal interrupts. Arduino Code for Idle Sleep Mode:

```
LowPower.idle(SLEEP_8S, ADC_OFF, TIMER2_OFF, TIMER1_OFF, TIMER0_OFF,
SPI_OFF, USART0_OFF, TWI_OFF);
```

Standard Low Power Example:
This is the simplest way of implementing the Low Power mode. It will use the LED as an indicator to tell if the device is in an active state or sleep state. The device will be in a sleep state for 5 seconds.

```
#include "ArduinoLowPower.h"

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); }

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
  LowPower.sleep(5000);
}
```

Here we can also change the line of the code to perform the Deep Sleep mode of the device.

```
//LowPower.sleep(5000);
LowPower.deepSleep(5000);
```

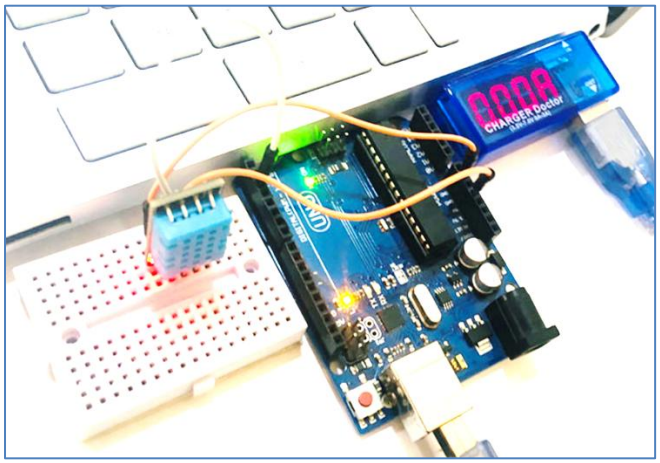
Low Power-Power Down Mode Example:
In this sketch, the Arduino blinks an LED for 2 s and is then powered down for 2 s, and during that time the ADC and Brown-Out Detect (BOD) are disabled. When powered down, the Arduino's current drops from several mA to less than a mA! We can use a USB ammeter to measure the current. From there, we can compute the power using the formula $P = VI$, where V is the supply voltage of 5 V.

```
#include "LowPower.h"

void setup() {
        pinMode(8, OUTPUT);
        }

void loop() {
        digitalWrite(8, HIGH);
        delay(2000);
        digitalWrite(8, LOW);
        LowPower.powerDown(SLEEP_2S, ADC_OFF, BOD_OFF);
        }
```

Let’s load this sketch onto our Arduino, which is running with a 5 V supply at 16 MHz. To see how little current is needed in sleep mode, we are using an Arduino ATmega328P board to minimize the current. In this setup, to demonstrate Arduino Power Save or Deep Sleep Modes, the Arduino is plugged into the USB ammeter. Then the USB ammeter is plugged into the USB port of the laptop. USB ammeter is a plug-and-play device used to measure the voltage and current from any USB port. The dongle plugs in between the

USB power supply (computer USB port) and the USB device (Arduino). This device has a 0.05 Ω resistor in-line with the power pin through which it measures the value of the current drawn. The device comes with four seven-segment displays, which instantly display the values of current and voltage consumed by the attached device. These values flip in an interval of every 3 seconds.

USB Ammeter



Connection of Arduino with the USB and Laptop

Arduino Code for Power-Down Periodic Mode:

```
LowPower.powerDown(SLEEP_8S, ADC_OFF, BOD_OFF);
```

The code is used to turn on the power-down mode. By using the above code, the Arduino will go to sleep for eight seconds and wake up automatically. We can also use the power-down mode with an interrupt, where the Arduino will go to sleep but only wakes up when an external or internal interrupt is provided.

Arduino Code for Power-Down Interrupt Mode:

```
void loop() {
   // Allow the wake-up pin to trigger an interrupt on low.
   attachInterrupt(2, wakeUp, LOW);
   LowPower.powerDown(SLEEP_FOREVER, ADC_OFF, BOD_OFF);
   // Disable external pin interrupt on the wake-up pin.
   detachInterrupt(2);
   // Do something here
}
```

In the void setup function, we have initiated the serial communication by using Serial.begin(9600), here the 9600 is the baud rate. We are using Arduino's built-in LED as an indicator for Sleep Mode. So, we have set the pin as OUTPUT and sent a LOW signal to this pin using the digitalWrite() command.

Before enabling the sleep mode, we are printing "Arduino:- I am going for a Nap" and making the built-in LED LOW. After that, Arduino sleep mode is enabled by using the command mentioned below in the code. The following code enables the idle periodic sleep mode of the Arduino and gives a sleep of eight seconds. It turns the ADC, Timers, SPI, USART, and 2-wire interface into the OFF condition. Then, it automatically wakes up Arduino from sleep after 4 seconds and prints "Arduino:- Hey I just Woke up".

So, by using this code Arduino will only wake up for 36 seconds (6 s×6 times of the loop) in a minute and will remain in sleep mode for the rest of the 24 (= 60 – 36 = 4 s×6 times of the loop) seconds, which significantly reduces the power consumed by the Arduino-based LED ON-OFF system. Therefore, if we use Arduino with Sleep Mode, we can increase the device runtime.

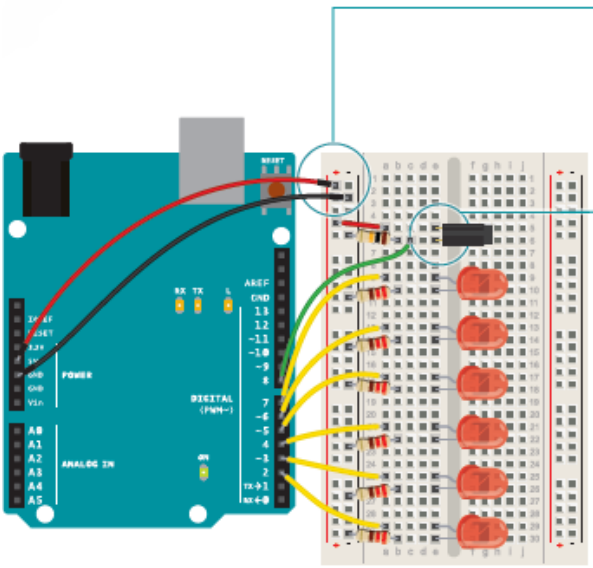
```
#include <LowPower.h>

#define LED_RED 8

void setup() {
  Serial.begin(9600);
  pinMode(LED_RED, OUTPUT);
  // SMCR = 0b00001101;
}


void loop() {
        digitalWrite(LED_RED, HIGH);
        delay(2000);
        Serial.println("Arduino:- I am going for a Nap");
        digitalWrite(LED_RED,LOW);
        LowPower.idle(SLEEP_4S,     ADC_OFF,     TIMER2_OFF,     TIMER1_OFF,
        TIMER0_OFF, SPI_OFF, USART0_OFF, TWI_OFF);
        Serial.println("Arduino:- Hey I just Woke up");
        delay(4000);
        }
```

Experimental Setup and Procedures:

[A]. LED Pattern:

1. Connect power and ground to your breadboard as shown in Fig. 1.
2. Connect the anode (longer leg) of six LEDs to digital pins 2-7 and connect the LEDs to the ground through 100 Ω resistors.
3. Connect one lead of the tilt switch to +5 V. Connect the other lead to a 10 kΩ resistor to the ground and connect the junction where they meet to digital pin 8.
4. Take the photos at different stages of LEDs ON/OFF states.

Experimental Setup:



1) Connect power and ground to your breadboard
2) Connect the anode (longer leg) of six LEDs to six digital pins 2-7 and connect cathodes of all LEDs to the ground through six 100 Ω resistors.
3) Connect one lead of the tilt switch to the +5 V and connect the other terminal of the tilt switch to the ground through a 10 kΩ resistor. Connect the junction of the switch and resistor to the digital input pin 8.

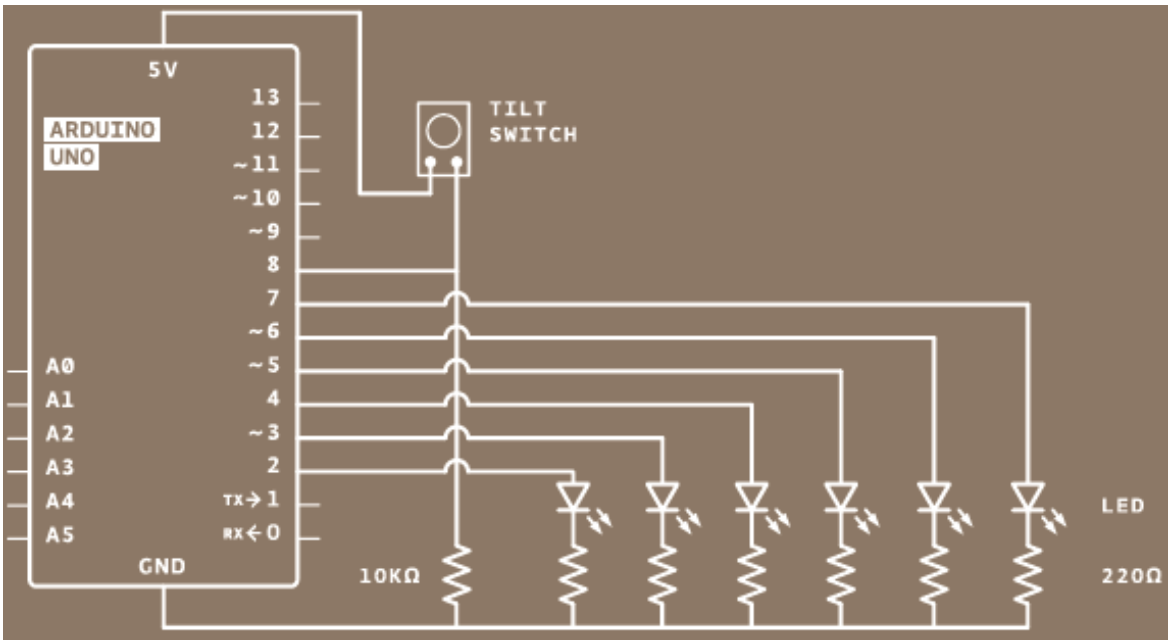


Fig. 1 Experimental Setup of an LED Light Pattern Control System using a Tilde switch and an Arduino Microcontroller Board.

Apparatus:

1) Arduino IDE (2.0.1 or any recent version)
2) Arduino Microcontroller board
3) LED lights (Red, Green, and Yellow- each 2)
4) Six 100 Ω resistors and one 10 kΩ resistor
5) One tilt switch
6) Jumper wires

Experimental Procedure:

The main task of this experiment is to implement an LED light pattern control system using a tilt switch. Connect the circuits as per the diagram of Fig. 1. Then plug in the Arduino microcontroller board to the PC.

Using Arduino IDE to write code:

1. Open the Arduino Uno IDE 2.0.1 (version may have been updated automatically on your computer) and a blank sketch will open. The window as in Fig. 2 will come up on your PC:

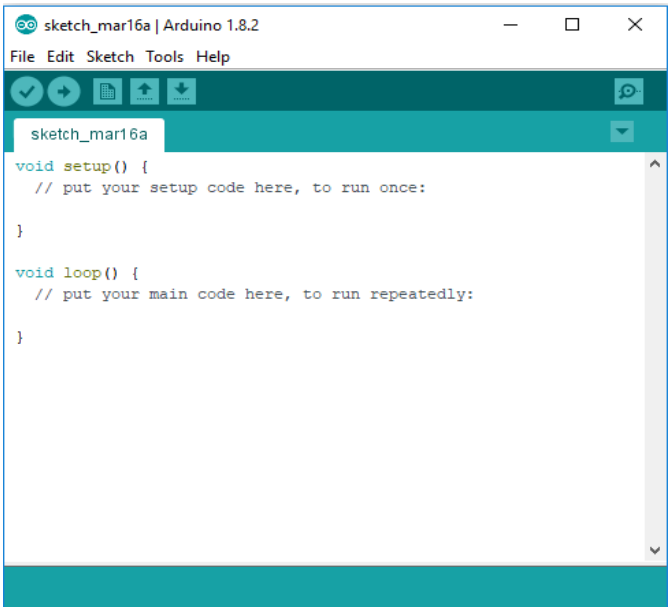


Fig. 2 IDE Environment (text editor)

2. Now, write the program by following the code writing steps in sequences given below on the blank sketch for the LED pattern control.
   a) Declare a named constant:
   You need several global variables in your program to get this all working. To start, create a variable named *SwitchPin* where the tilt switch would be connected.
   b) Create a variable to hold the time
   Create a variable of name **unsigned long** to hold the time of the LED that was changed last time.
   c) Declare variable names for the inputs and outputs
   Create two variables for the current and previous switch states to compare the switch positions from one loop to the next. Create another variable named *led* to store the pin numbers of the LEDs to be stored to determine which one to be turned on next time. Start out with pin 2.
   d) Declare a variable describing the interval between events
   The last variable you need to create is the interval between each LED turning on. This will be a **long** data type. In 10 minutes (i.e., the time between each LED turning on), 6,00,000 milliseconds (1 minute = 60 seconds and 1 second = 1000 milliseconds) pass. If you want the delay between lights to be longer or shorter, this is the number you change.
   e) Set the direction of your digital pins
   In the **void setup()** function, you need to declare the LED pins 2-7 as the output pins. A **for** loop declares all six pins as OUTPUT (so that the microcontroller can send signals to these pins) with just 3 lines of code. You also need to declare *SwitchPin* as an INPUT (so that the microcontroller can receive a signal from this pin).
   f) Check the time since the program started running
   When the **void loop()** starts, you are going to get the amount of time the Arduino has been running with **millis()** and store it in a local variable named *CurrentTime*.
   g) Evaluate the amount of time that has passed since the previous loop()
   Using an **if()** statement, you'll check to see if enough time has passed to turn on an LED. Subtract the *CurrentTime* from the *PreviousTime* (initialized as 0 for the first time

during declaration) and check to see if it is greater than the interval variable. If 6,00,00 milliseconds have passed (one minute), you'll set the variable *PreviousTime* to the value of *CurrentTime*.

h) Turn on an LED, prepare for the next one
PreviousTime indicates the last time an LED was turned on. Once you have set *PreviousTime*, turn on the LED, and increment the *led* variable, for example from 2 to 3. The next time you pass the time interval, the next LED will light up.

i) Check to see if all lights are on
Add one more *if* statement in the program to check whether the LED connected to pin 7 (next pin) is turned on or not. was turned on. However, don't do anything with this yet, wait for the end of an hour (set time) to decide.

j) Read the value of the switch
After checking the time, you should check the switch state by reading it and storing the value in the *SwitchState* variable.

k) Reset the variables to their defaults if necessary
With an *if* statement, check the position or status of the switch. The != evaluation checks to see if *SwitchState* variable doesn't equal the *PrevSwitchState* variable. If they are different, turn the LEDs off, return the *led* variable to the first pin, and reset the timer for the LEDs by setting *PreviousTime* to the value of *CurrentTime*.

l) Set the current state to the previous state
At the end of the **void loop()** function, save the value of the *SwitchState* variable in the value of the *PrevSwitchState* variable so that you can compare it to the value you get for the *SwitchState* variable in the next **void loop()**.

Code of a LED light pattern control system with millis() function:

```
const int SwitchPin = 8;
unsigned long PreviousTime = 0;
int SwitchState = 0;
int PrevSwitchState = 0;
int led = 2;
long interval = 60000; // for 1 min = 60,000 ms delay

void setup() {
for (int x = 2; x < 8; x++) {
pinMode(x, OUTPUT);
}
pinMode(SwitchPin, INPUT);
}

void loop() {
unsigned long CurrentTime = millis();
if (CurrentTime - PreviousTime > interval) {
PreviousTime = CurrentTime;

digitalWrite(led, HIGH);
led++;
if (led == 7){
}
}
```

```
SwitchState = digitalRead(switchPin);
if (SwitchState != PrevSwitchState){
for (int x = 2 ; x < 8; x++) {
digitalWrite(x, LOW);
}
led = 2;
PreviousTime = CurrentTime;
}
PrevSwitchState = SwitchState;
}
```

3. After writing the program you must save your code file in a folder. For this purpose, please go to the File Menu → Save As → give a File name in the file name box (for example, Traffic_light_control) → Select Save.
   N.B. After saving your code, a sketch file with the desired file name will be stored in a folder with the same file name.
4. Now you need to verify/compile your code to find out and correct the errors. For this, please go to the Sketch Menu → Verify/Compile or press Ctrl+R or click on this button just below the menu bar. If the code is correct then the 'compilation done' message will be displayed, otherwise, an error message will be displayed.
5. After the compilation is done successfully, you need to upload your code onto the Arduino Uno board. To upload the program, connect your Arduino Uno R3 board to your PC with a USB cable. Before uploading the code, select the board type and port at your Arduino IDE. For this purpose, please go to the Tools Menu → Board: "Arduino Uno" → Arduino Uno. Then again go to the Tools Menu → Ports → COMx, here x means an integer number that corresponds to the Arduino Uno board that is automatically detected by the IDE.
6. After you have selected the board and port, you must upload the code. For this, please go to the Sketch Menu → Upload or press Ctrl+U or click on this button just below the menu bar. If the code is correct then the code upload-done message will be displayed, otherwise, an error message will be displayed. Based on the error message (if any), please go to the code, and correct them. Usually, students forget to set up the Tools or select an inappropriate board or COMx port number to get such kinds of error messages.

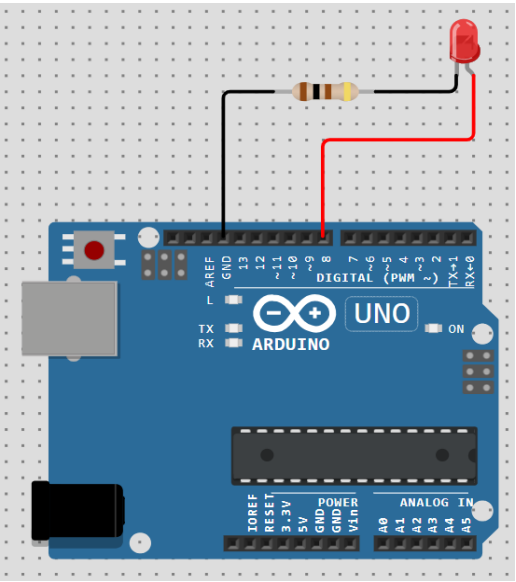
[B]. Power Saving Calculation:

1. Connect power and ground to your breadboard as shown in Fig. 2.
2. Connect the anode (longer leg) of red LED to digital pin 8 and connect the cathode (shorter leg) of red LED to the ground through a 100 Ω resistor as shown in Fig. 2.
3. Connect one USB ammeter to the USB port of the laptop or desktop PC and connect the Arduino Board to the USB ammeter as shown in the theory section.
4. Connect one leg of the switch to the +5V port of the Arduino Board. Connect the other leg to the ground through a 10 kΩ resistor and connect the junction to pin 2.
5. Measure the current drawn by the system during the ON, OFF, and sleep times. Compute the power and energy saved in different stages.

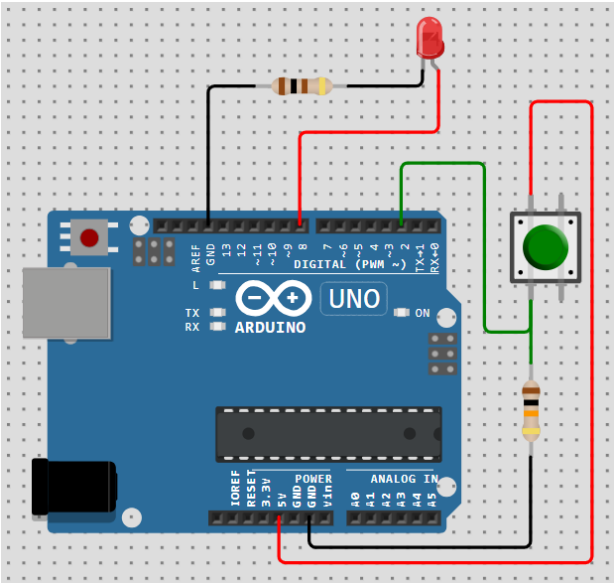
Apparatus List:

1. Arduino Board
2. Breadboard
3. Resistors
4. Switches
5. LED

6. USB Ammeter
7. Jumper Wires
8. Laptop or PC with Arduino IDE installed in it.



(a) LED state change without any switch;



(b) LED state change with switch:

Fig. 2 Power Measurement during different states of LED

Questions for report writing:

1) Include all codes and scripts in the lab report following the lab report writing template.
2) Show the output/results in the form of images. Give their captions and descriptions.
3) Configure the system to have delays for outputs according to your ID. Consider the last six digits from your ID (if your ID is XY-PQABC-Z then consider P s delay for the 1st LED and Z s delay for the last LED). In case any digit is zero then use Z s delay by default. Include the program and results within your lab report.
4) Recreate this program to turn ON LEDs in one direction when the input switch is HIGH and reverse the direction of turning ON the LEDs when the input switch is LOW.
5) Include the **Proteus simulation** of the given program and the program of Question # 4. Explain the simulation methodology.
6) Implement the other program given in theory section in **Proteus** and show the simulation outcomes through screen shots.
7) **Compute power saving in different modes** of the program.

Reference(s):

[1] https://www.arduino.cc/.
[2] ATMega328 manual
[3] https://www.avrfreaks.net/forum/tut-c-newbies-guide-avr-timers
[4] http://maxembedded.com/2011/06/avr-timers-timer0/